

(PAL) employ an LLM to understand issues and build programmes. The solution phase is delegated to a runtime, such as a Python interpreter.

Large language models (LLMs) are a sort of artificial intelligence (AI) that are used by program-aided language models (PAL) to read issues written in natural language and produce programmes as intermediate stages in the reasoning process. To solve the issue, the resulting programme is then run in a runtime environment, such as a Python interpreter.

For jobs requiring programmatic thinking, such as coding, programming, and software development, PAL is particularly helpful. It allows people to speak naturally about an issue, and it then creates the associated programmable solution. The time and effort needed to design software, especially for common activities, can be reduced with the use of this method.

PAL is still a young technology, therefore there are still many of obstacles to solve, such making sure the created programmes are accurate and effective. Additionally, not all programming assignments may be ideal for PAL, especially those that need for a lot of originality or imagination. However, PAL has the potential to completely alter how we approach programming and software development.

Example

Problem: Create a program that generates a random name for a new company.

The user inputs the problem description in natural language: "Create a program that generates a random name for a new company."

The large language model (LLM) in PAL reads and analyzes the problem description and generates an intermediate program as a series of runnable steps:

Define a list of adjectives and a list of nouns

Use a random number generator to select an adjective and a noun from the corresponding lists

Combine the selected adjective and noun to form a company name

Print the company name as the output

The generated program is then handed off to a runtime environment, such as a Python interpreter, which executes the program to produce the desired output:

```
adjectives = ['Blue', 'Green', 'Red', 'Yellow', 'Orange', 'Purple']
```

```
nouns = ['Company', 'Enterprises', 'Solutions', 'Industries', 'Technologies', 'Innovations']
```

```
adjective = random.choice(adjectives)
```

```
noun = random.choice(nouns)
```

```
company_name = adjective + ' ' + noun
```

```
print(company_name)
```

Output

Red Enterprises

Yellow Technologies

6. ReAct

LLMs are employed in the ReAct framework to provide interspersed reasoning trajectories and task-specific actions.

The model can induce, track, and escalate action plans using logic trails, and it can even manage exceptions.

The action phase enables the user to interact with and acquire data from other sources, such as surroundings or knowledge bases.

ReAct enables LLMs to communicate with external tools to retrieve more data, resulting in more trustworthy and factual replies.

The ReAct architecture uses LLMs to produce interleaved reasoning traces and task-specific actions, enabling the model to create, monitor, and modify action plans as well as manage errors. ReAct enables the model to engage with other tools and acquire information that results in more trustworthy and factual replies by utilising the model's capabilities in connection with external sources, such as knowledge bases or surroundings. With this strategy, the model may make the most of its aptitude for reasoning and natural language processing while also adding outside data to increase precision and performance on certain tasks.

Examples

Prompt : Problem: Given a natural language question, generate a response that includes a list of the top 5 most relevant articles on the topic.

The user inputs the problem description in natural language: "What are the top articles on machine learning?"

Output :

The ReAct framework reads and analyzes the problem description and generates an action plan as a series of interleaved steps:

Use a language model to understand the natural language question and generate a list of relevant keywords

Interface with an external search API, such as Google or Bing, to search for articles related to the keywords

Use a ranking algorithm, such as PageRank, to rank the articles based on relevance

Select the top 5 articles based on the ranking and format the response in natural language

The generated action plan is then executed using a combination of LLMs and external tools to produce the desired output:

```
import openai
```

```
import requests
```

```
# Use OpenAI's GPT-3 to generate a list of relevant keywords
```

```
keywords = openai.Completion.create(
```

```
engine="davinci",
```

```
prompt="What are the top articles on machine learning?",
```

```
max_tokens=100,
```

```
n=1,
```

```
stop=None,
```

```
temperature=0.5,
```